

CMPSC 473 – Project 1

Team Members: Yogeshvar Reddy Kallam(yvk5381) Lakshmi Sivani Devarapalli(lfd5379)

Introduction

We employed our scheduler as a discrete-event, single-CPU simulator that advances a logical clock (global_time) tick by tick. All shared by our state (the thread table, queues, device status) is protected by a single global mutex so all scheduling decisions will be atomic and race-free.

Our synchronization code was a significant area of focus for us. We used a per thread condition variable so that exact, precise wakeups could be achieved with pthread_cond_signal. This allows the scheduler to wake only the one, lone thread that's ready to run, which is much more efficient than waking all threads. We also used a global condition variable for system-state transitions, such as when we needed to "fast forward" time when the CPU was idle. The third system mimics a thread's life cycle from NEW to READY to RUNNING to BLOCKED to TERMINATED and supports three various scheduling policies, selectable on startup such as - FCFS which is Non-preemptive, SRTF which is Preemptive, MLFQ which is a preemptive priority-based scheduler.

Core Data Structures We Used

thread_tcb_t (Thread Control Block): This was our primary data structure for each thread. We designed it to hold all metadata our scheduler would need : tid, state, its personal cond variable, arrival_time, io_finish_time, rem_srtf_time for SRTF, and additional fields for MLFQ to track its queue level and ticks_used_in_quantum.

Queues (queue_t): We implemented a standard singly-linked list queue structure. We used this for the main ready_queue, the io_queue, each of the five MLFQ-level queues, and each semaphore's wait_queue.

Semaphores (semaphore_t): For semaphores, we created a simple struct containing an integer val and its own wait_queue to manage blocked threads.

Design and Architecture

Initial Idea (The "Broadcast" Model), We first tried what we thought was the simplest approach for FCFS, using a single global condition variable and pthread_cond_broadcast. We thought this "fire alarm" approach would be safe, as every woken thread could then re-evaluate its condition.

This broadcast model failed immediately and caused a thundering herd effect where too many threads woke up unnecessarily. We faced deadlocks because our state management was flawed. Threads weren't being correctly re-enqueued after running for a tick. This led us to our first major backstep. We decided to leave the broadcast model and retry our scheduler to use the more precise `pthread_cond_signal` with per-thread condition variables.

Our new signal-based design worked for continuous CPU tasks, but we faced our biggest challenge when we introduced I/O. We saw that our model would deadlock if the CPU went idle while an I/O operation was in progress. Because our logical clock, `global_time`, was only advanced by CPU ticks, the clock would freeze. The thread waiting for I/O would wait forever for a time that would never come. This was our most difficult struggle.

After many debugging sessions, our breakthrough came when we designed and implemented a "time jump" mechanism. We wrote logic to detect when the CPU is idle but future events (like an I/O completion) are pending. In this case, our scheduler fast-forwards `global_time` directly to that event's timestamp. This FCFS code, which we developed together, solved the deadlock and became our stable baseline.

The SRTF Refactoring Challenge, where Integrating SRTF was much harder than we anticipated. We had to inject preemption logic at every point a thread could become ready (initial arrival, finishing I/O, being unblocked by a semaphore). Replace every FCFS-specific decision point (e.g., "signal the head of the queue") with new, "policy-aware" blocks that could either pick the first-in-line or search the entire queue for the shortest job.

Implementing MLFQ felt like building a new scheduler on top of our existing one. Our initial "check-if-I-am-best" wait loop for MLFQ led to a subtle but critical deadlock. Multiple threads could believe they were the best, but none would actually take the step of assigning the CPU. Our final breakthrough was to simplify the logic where the first thread to find `cpu_thread == NULL` under the lock takes the responsibility of atomically dequeuing the best thread (from the highest-priority queue) and assigning it to `cpu_thread`. This simple, authoritative assignment rule resolved the deadlock completely.

Team Workload Distribution

We developed this project using a highly collaborative, pair programming style. The process of designing the architecture, especially for solving the major deadlocks, was a joint effort. Usually, one of us would write the initial code for a new feature (like the SRTF preemption logic or the MLFQ data structures), and the other would review it while simultaneously debugging issues while working on test cases. The debugging sessions for the most difficult bugs were always done together, tracing the program's execution step-by-step to find the root cause. This shared approach ensured we both understood all parts of the final codebase.